SQL, MySQL y PL/SQL

Consultas, relaciones, transacciones y programación en Oracle

CódigoClaro · Material didáctico de apoyo · Edición octubre de 2026

Cómo estudiar este material

Lee el concepto, escribe los ejemplos sin copiar y modifica los casos de prueba. Los fragmentos son ejemplos didácticos: adapta nombres, tipos y validaciones al enunciado real.

1. Modelo relacional

Una base de datos relacional organiza datos en tablas. Cada fila representa un registro y cada columna un atributo. Una clave primaria identifica cada fila; una clave foránea mantiene la relación con otra tabla. Normaliza para reducir duplicidad y anomalías.

```
CREATE TABLE alumno (
  id INT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  email VARCHAR(120) UNIQUE
);
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  modulo VARCHAR(80) NOT NULL,
  PRIMARY KEY (alumno_id, modulo),
  FOREIGN KEY (alumno_id) REFERENCES alumno(id)
);
```

2. Consultas SELECT

```
SELECT nombre, email
FROM alumno
WHERE nombre LIKE 'A%'
ORDER BY nombre ASC;

SELECT modulo, COUNT(*) AS total
FROM matricula
GROUP BY modulo
HAVING COUNT(*) >= 2;
```

WHERE filtra filas antes de agrupar; HAVING filtra grupos. DISTINCT elimina duplicados del resultado. Usa nombres de columnas explícitos y parámetros para datos introducidos por usuarios.

3. JOIN y subconsultas

```
SELECT a.nombre, m.modulo
FROM alumno a
INNER JOIN matricula m ON m.alumno_id = a.id;

SELECT nombre FROM alumno
WHERE id IN (SELECT alumno_id FROM matricula);
```

INNER JOIN devuelve coincidencias. LEFT JOIN conserva todas las filas de la tabla izquierda y completa con NULL cuando no hay coincidencia. Comprueba cardinalidad para no duplicar resultados inesperadamente.

4. INSERT, UPDATE y DELETE

```
INSERT INTO alumno (id, nombre) VALUES (1, 'Luz');
UPDATE alumno SET nombre = 'Lucía' WHERE id = 1;
DELETE FROM alumno WHERE id = 1;
```

Antes de ejecutar UPDATE o DELETE, verifica el WHERE con un SELECT. En aplicaciones reales usa consultas parametrizadas para evitar inyección SQL.

5. Transacciones e índices

Una transacción agrupa operaciones que deben completarse de forma coherente. ACID significa atomicidad, consistencia, aislamiento y durabilidad. COMMIT confirma y ROLLBACK revierte cambios pendientes. Los índices aceleran lecturas concretas, pero consumen espacio y encarecen escrituras.

```
BEGIN;
UPDATE cuentas SET saldo = saldo - 50 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 50 WHERE id = 2;
COMMIT;
```

6. PL/SQL básico (Oracle)

```
DECLARE
  v_total NUMBER := 0;
BEGIN
  SELECT COUNT(*) INTO v_total FROM alumno;
  DBMS_OUTPUT.PUT_LINE(v_total);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(SQLERRM);
    RAISE;
END;
/
```

DECLARE contiene variables; BEGIN contiene sentencias ejecutables; EXCEPTION gestiona errores. SELECT INTO debe devolver una sola fila; para múltiples filas estudia cursores. En código de producción no ocultes errores capturando WHEN OTHERS sin volver a lanzar o gestionar adecuadamente.

7. Procedimientos, funciones y triggers

Un procedimiento ejecuta una operación y puede usar parámetros IN/OUT. Una función devuelve un valor. Un trigger se ejecuta ante un evento como INSERT, UPDATE o DELETE; evita lógica excesiva que dificulte comprender los cambios.

```
CREATE OR REPLACE FUNCTION duplicar(p_num NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN p_num * 2;
END;
/
```

Ejercicios de autoevaluación

• Lista los alumnos que no tienen matrículas usando LEFT JOIN.

• Calcula la media de notas por módulo con GROUP BY y HAVING.

• Escribe un bloque PL/SQL que recorra un cursor y muestre cada nombre.

- Explica cuándo usar una transacción y por qué un índice no siempre mejora el rendimiento.